

Solving sudoku with reasoning, evolution and deep learning

Matúš Halák (2724858), Thibault Giesbertz (2791192)

November 2025

Abstract

Sudoku is a logical puzzle that requires solver algorithms to navigate its large search space. In this work, we explore symbolic reasoning, evolutionary algorithms, and deep learning approaches to this problem. We evaluated (and trained) our algorithms on the sudoku-extreme dataset, as well as, smaller datasets related to the non-consecutive sudoku variant. Our results show that symbolic reasoning, and in particular our efficient CDCL implementation, remains the most accurate and consistent approach, outperforming evolutionary and deep learning methods across all tested instances. While symbolic reasoners remain the go-to solution for easily achievable high sudoku performance, the evolutionary and deep learning alternatives explored in this paper demonstrate clear potential. Future research should investigate hybrid algorithms designed to mitigate the respective weaknesses of the individual approaches.

1 Introduction

Games and puzzles have long been used as testing ground for AI, since they provide clear rules and enable objectively evaluating the performance of an AI system on a typically human task. Sudoku is an example of a popular logic-based puzzle and a classical constraint satisfaction problem (CSP) that has been extensively studied by mathematicians and computer scientists. In its classical formulation on a 9×9 board, each row, column, and each 3×3 box of a valid sudoku solution must contain digits 1-9 exactly once. Because of its popularity, sudoku puzzles have been extended to bigger boards (16×16 , 25×25) and to different rule-sets that add novel constraints to the puzzle. One such addition, which we briefly explore in this report is the non-consecutive rule, which states that numbers in neighboring cells cannot differ by 1. Despite a deceptively simple rule-set, sudoku has attracted scholarly attention, because each well-posed sudoku puzzle has a single unique solution, finding which requires navigating an astronomically large search space. While humans can solve sudoku

puzzles with relative ease, AI systems can struggle with navigating large search spaces. In particular, vanilla sudoku with 81 cells, each with 9 options has 9^{81} possible board configurations, and this number explodes for bigger board configurations. While the problem constraints narrow down the sudoku search space substantially, it remains extremely large and poses an interesting challenge for AI algorithms.

Although sudokus can be solved with brute-force search algorithms, these approaches can not scale to bigger board sizes and are therefore not discussed further in this report. Instead we focus on approaches that intelligently navigate the search space. For instance, algorithms designed to solve the Boolean satisfiability (SAT) problem, known as SAT solvers have successfully been applied to sudoku and many other CSPs. SAT solvers leverage the logical nature of sudoku, by encoding all possible values for each cell into propositional variables, and encoding the constraints into logical clauses between the propositional variables. SAT solvers then derive the solution through a logical reasoning algorithm which assigns true/false to the propositional variables, most famously the DPLL algorithm (discussed in Section 2). A general issue with SAT solvers is that their worst-case complexity is $\mathcal{O}(2^n)$ where n is the number of propositional variables (which scales cubically with sudoku size). While modern SAT solvers have been highly optimized to avoid this worst-case complexity as much as possible, a naive SAT solver implementation remains impractical for hard or large sudoku problems.

In this report, we wanted to investigate various approaches to solving sudoku and compare their strengths and weaknesses¹. First, we implemented a more advanced SAT solving algorithm (CDCL) and compared it to a naive implementation of the seminal DPLL SAT solving algorithm, to investigate how sudoku might be solved through reasoning approaches based on logic. Next, we wanted to see whether the logical framing of the problem was necessary, and whether other optimization techniques were better suited for exploring the sudoku search space. To this

¹All code, including a detailed README file is available at github.com/matushalak/kr2025

end, we developed an evolutionary algorithm that efficiently explores full-board solutions directly, without the need for a logical encoding, using a rule-aware fitness function to guide the search (Section 3). Finally, we wondered whether the rules, as well as, the solution strategies to the game of sudoku could be learned by a neural network purely through exposure to a large number of sudoku puzzles. We built a custom transformer model, hoping to leverage the attention mechanism and capture global relationships between distant elements in the puzzle (Section 5). We hypothesized that the CDCL algorithm will outperform DPLL, because of the advanced techniques it uses to guide the variable assignment process. Our investigation of evolutionary algorithms and neural networks was exploratory, although we expected both approaches to outperform the SAT solvers on run-time, at the cost of imperfect accuracy.

2 Reasoning through sudoku with SAT solvers

2.1 Problem formulation

Sudoku can be naturally viewed as constrain satisfaction problem whose structure translate well into Boolean encoding. Following the classical formulation by Lynce and Ouaknine [1], we represent each possible assignment of digit $v \in \{1, \dots, N\}$ to the cell at row r and column c by a propositional variable

$$\text{var}(r, c, v) = rN^2 + cN + v.$$

A complete sudoku board corresponds to a truth assignment to these N^3 variables that jointly satisfies the logical constraints of the puzzle. We encode: first that each cell contains exactly one digit, second that each digit appears exactly once per row, column, and box, and third that the fixed clues in the puzzle must hold. These constraints are expressed as families of CNF clauses, resulting in an encoding structure similar to earlier SAT-based sudoku solvers [1, 2]. Because the encoding is fully modular, we can seamlessly extend it to additional rule-sets.

For the *non-consecutive* sudoku variant, we add pairwise constraints between adjacent cells, ruling out assignments that differ by 1. Importantly, the entire extension requires no modification of the underlying solvers, only a additional selection of clauses. This allows us to evaluate DPLL and CDCL methods on the same logical formulation.

2.2 DPLL algorithm

The Davis–Putnam–Logemann–Loveland (DPLL) algorithm is the classical complete SAT procedure and forms a natural conceptual baseline. DPLL recursively chooses an unassigned literal, branches on its value, and closes off branches via unit propagation. Our implementation uses the MOM heuristic to prioritize literals from short clauses, which is a traditional way of introducing a small amount of guidance into branching.

However, DPLL lacks most of the algorithmic features that define modern SAT solvers. In particular, it does not learn new clauses from conflicts, nor can it backtrack non-chronologically. As a result, DPLL repeatedly explores similar failing regions of the search tree resulting in it exploring exponentially large inconsistent subspaces.

2.3 CDCL algorithm

Conflict-Driven Clause Learning (CDCL) extends DPLL with several innovations that fundamentally change its behavior. Our CDCL implementation captures the core of the architecture described in the SAT Handbook [3]. The solver maintains watched literals for efficient clause evaluation, uses Boolean constraint propagation supplemented by binary clause implications, and employs VSIDS-style variable activity scores together with phase saving. Importantly, upon encountering a contradiction the solver performs a 1-UIP conflict analysis, learns a new clause summarizing the conflict, and backjumps directly to the relevant decision level. This is further stabilized by periodic Luby restarts, which help the solver escape regions of the search space with insufficient propagation.

Although our solver is greatly simplified compared to industrial SAT tools, these ingredients are enough to capture the characteristic efficiency of the CDCL approach. That being: the search space collapses quickly under clause learning, and repeated conflicts betters the solver’s ”understanding” of the structure of the puzzle. The effectiveness of our CDCL approach was also confirmed by our third place ranking during our Knowledge Representation class’ SAT solver competition. (solved 261/400 puzzles).

2.4 Results

CDCL solved all puzzles with consistently low run-times and remarkably low variance. The solver typically required well under a second per puzzle, often terminating after only a few hundred propagation steps. DPLL also solved all puzzles but its run-

time distribution clearly illustrates the characteristic spikes of deep recursive search. Even with MOM branching, DPLL frequently diverged toward exponentially large branches, confirming prior findings on naive backtracking solvers.

3 Evolving sudoku solutions with evolutionary algorithms

3.1 Problem formulation

Given the limitations of purely symbolic approaches, we were curious whether stochastic search in the *solution* space might offer a fundamentally different route to solving sudoku. Evolutionary algorithms (EAs) have been applied to sudoku with some success [4], especially when combined with local repair heuristics. Their main attraction is that they require no logical encoding: instead of assigning truth values to propositional variables, the EA directly maintains a population of candidate sudoku boards. Because sudoku constraints can be incorporated into a fitness function, EAs are easily adapted to variants such as the non-consecutive rule. Prior work has demonstrated that preprocessing, hybrid mutation operators, and local search can dramatically improve EA performance on structured combinatorial puzzles [5].

3.2 Evolutionary algorithm

Our EA maintains a population of complete sudoku boards, where each row is initialized as a permutation of the missing digits respecting the original clues. The fitness function counts column and box conflicts, and optionally includes penalties for violations of the non-consecutive rule. Offspring are generated via row-wise uniform crossover, tournament selection, and elitism. To improve local refinement, we use a memetic mutation operator that performs conflict-guided swaps within rows, columns, or boxes, accepting moves that do not worsen fitness. This is similar to the hybrid local search strategies shown in previous work to accelerate EA convergence on sudoku [4, 6]. Optional hard restarts allow the algorithm to escape stagnation.

For our experiments, we used a population size of 500, tournament size $k = 3$, full crossover probability (1.0), and a mutation rate of 0.3. Each mutation attempt performed up to 40 conflict-guided swaps. Two elite individuals were preserved each generation. Runs were allowed up to 5000 generations, with early stopping triggered upon reaching zero conflicts. Restarts were disabled for the classical puzzles,

but enabled for the non-consecutive set, with a stagnation threshold of 1000 generations, since the non-consecutive puzzles gave more issues getting stuck at local minima. These settings produced stable convergence on most classical instances, but were insufficient for the more tightly constrained non-consecutive problems.

3.3 Results

On classical sudoku, the EA solved most puzzles within a reasonable number of generations. Its performance is competitive given the lack of explicit logical structure, and the generation boxplot in Figure 2 indicates robust convergence for many instances. Nonetheless, occasional stagnation occurs, which is typical for evolutionary optimization on highly constrained search spaces.

On the non-consecutive puzzles, the EA was substantially less successful. Despite restarts and extensive local search, the algorithm consistently became trapped in local minima. This is consistent with observations from earlier evolutionary studies: when constraints become strongly interdependent, local swaps that fix one region often break another, producing fitness landscapes that are difficult for mutation-based search to navigate [5]. The EA reached low conflict scores but never a valid solution.

4 Comparative Results (Reasoning vs Evolution)

To compare the DPLL, CDCL and EA approaches directly, we evaluated them on a shared set of 100 puzzles drawn from the sapientinc/sudoku-extreme test split (the same dataset used for the transformer neural network experiments in Section 5). For each puzzle, we recorded the runtime and whether the solver found the unique solution. Figure 1 shows the distribution of runtimes across the three methods, and Table 1 summarizes the success rates.

Method	Success rate
CDCL	100%
DPLL	100%
EA	92%

Table 1: Success rates on the classical sudoku test set (100 puzzles).

The evolutionary algorithm (EA) solved most puzzles (92/100). Its runtimes were relatively stable but generally slower than CDCL and even DPLL. Because the EA explores the space of full candidate boards,

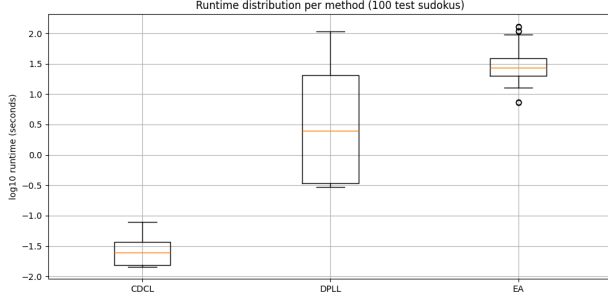


Figure 1: Distribution of runtimes for CDCL, DPLL, and the evolutionary algorithm (logarithmic scale).

its convergence depends on the structure of the fitness landscape rather than on guaranteed logical inference. Figure 2 shows the distribution of generations required for solved puzzles. In practice, the EA converged reliably on easier instances but occasionally stagnated, reflecting the limitations of purely stochastic optimization.

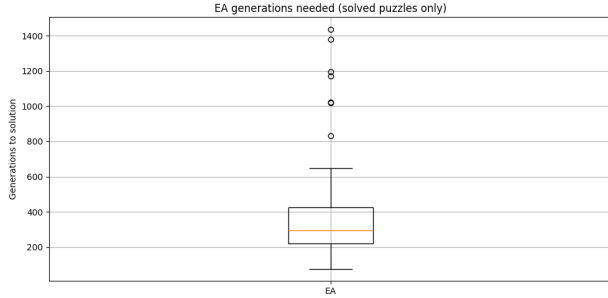


Figure 2: Amount of generation to solve the puzzles

4.1 Results on non-consecutive sudoku

The non-consecutive variant introduces an additional adjacency constraint: orthogonally neighboring cells must not contain digits that differ by 1. While this rule is simple to express at the CNF level, each prohibited pair is captured by a small set of binary clauses, it has a noticeable structural effect. The constraint increases the coupling between cells and creates longer chains of forced assignments. In the literature, similar effects have been reported for large and highly constrained Sudoku SAT encodings, such as 25×25 instances [7]. Because our encoder is modular, no changes to the SAT solvers were required: we simply activated the non-consecutive clause set.

Figure 3 shows the runtime distribution for the non-consecutive experiments, comparing CDCL with the

evolutionary algorithm. DPLL is omitted here because it did not terminate on a single instance within a reasonable time window, the additional adjacency constraints push basic backtracking beyond its practical limits.

CDCL, on the contrary, solved all the ten puzzles. Runtimes increased slightly relative to the classical case, but the solver remained stable: clause learning and non-chronological backtracking quickly absorbed the new constraints. As before, the combination of VSIDS scoring, watched literals and Luby restarts produced a search trajectory that remains robust even when the constraint graph becomes denser.

The evolutionary algorithm struggled noticeably. Even with restarts enabled and despite repeatedly reaching low conflict scores, it did not find a valid solution to any of the ten puzzles. This pattern matches earlier findings from evolutionary sudoku solvers [4, 5]: as local dependencies become tighter, mutation-based search is prone to becoming trapped in deep local minima. The non-consecutive rule amplifies this effect, since improving one region of the grid often disrupts another. Without a stronger repair mechanism or hybridization with symbolic inference, the EA lacks the capabilities to escape these fitness plateaus [6].

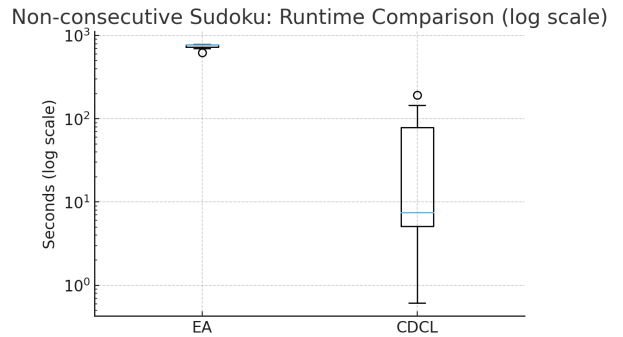


Figure 3: Runtime distribution for the non-consecutive sudoku experiments (EA vs. CDCL).

Method	Success rate
CDCL	100%
DPLL	Did not finish
EA	0%

Table 2: Success rates for the non-consecutive sudoku experiments (10 puzzles).

5 Learning sudoku with Transformer neural networks

While being impressed with the performance of EA’s on the sudoku problems, we wanted to investigate whether performing a separate search on each problem was at all necessary. Namely, we wondered - can an AI algorithm learn to capture the regularities in sudoku puzzles to the extent that it can immediately predict the solution to a previously unseen puzzle without performing any search on the possible moves? The class of AI algorithms naturally suitable for this challenge were deep neural networks, which are in theory able to learn arbitrarily complex mappings from input to output [8]. We reasoned that it could, in theory be possible to train a neural network that contained the mapping from any 9x9 sudoku to its solution within the network weights. The main motivation behind pursuing a Transformer neural network architecture for sudoku, was that Transformers are based on the powerful attention mechanism [9], which allows the network to capture relationships between all elements in the input. We reasoned that this could be useful for sudoku, because the digit prediction for any of the empty cells must be informed not just by its local neighbors, but by logical relationship between digits across the entire board.

At the same time, we knew the Transformer architecture was not designed to solve logical problems, and that teaching the network to solve sudoku puzzles without performing any search, backtracking or constraint propagation, was a very difficult task. To succeed, the network would have to learn both the sudoku constraints and the solution strategies at the same time. In the upcoming sections we discuss our network architecture and training strategy to tackle this problem in detail. We also realized that training a Transformer would require significant amount of training data. For this reason, we focused on the classical 9x9 sudoku formulation, for which we were able to find a Hugging Face data set of 4.223 Million well-posed sudoku problems ranging from easy to hard puzzles [sapiointnc/sudoku-extreme]. Because of time and compute constraints, we used small subsets of this dataset, but we could easily adjust the number of training samples by orders of magnitude.

5.1 Problem formulation

For a sudoku problem of size s (we used 9x9 sudoku, $s = 9$), we define a sudoku puzzle as a vector $\mathbf{x} \in \{0, 1, 2, \dots, s\}^{s^2}$ containing the clue digits 1- s from the $s \times s$ sudoku board flattened in row-major order, and empty cells containing 0. A solution vector

$\mathbf{y} \in \{1, 2, \dots, s\}^{s^2}$ contains the uniquely-solved grid in row-major order. We further define our Transformer neural network $F_\theta(\mathbf{x}) : \{0, 1, 2, \dots, s\}^{s^2} \rightarrow \mathbb{R}^{s^2 \times s}$, that outputs for each of the s^2 sudoku cells, a probability distribution over s candidate digits. At test time, the predicted solution vector $\hat{\mathbf{y}}$ is obtained by $\hat{\mathbf{y}} = \arg \max F_\theta(\mathbf{x})$. The objective is to learn model parameters θ such that $\hat{\mathbf{y}} = \mathbf{y}$ for previously unseen puzzles. We measure both full-board accuracy and per-cell accuracy. While the model produces predictions for all board positions, only predictions on the empty cells are used to measure performance.

5.2 Transformer Network architecture

Our Transformer neural network F_θ consists of L layers of transformer encoder blocks f_{θ_i} stacked one after another, followed by a single linear layer that produces for each cell a probability distribution over s candidate digits.

Sudoku embeddings Before applying the transformer encoder blocks in sequence, we once map the original input sequence of s^2 discrete symbols / tokens (we will refer to the s^2 sequence length with T for number of tokens) $\mathbf{x} \in \{0, 1, 2, \dots, s\}^T$ to a high-dimensional continuous representation (embedding) $\mathbf{E} \in \mathbb{R}^{T \times d_{model}}$, where d_{model} represents the embedding dimension of the model. All our transformer encoder blocks take as input such a $T \times d_{model}$ matrix and through multi-head attention produce another $T \times d_{model}$ matrix that can be fed into other transformer blocks or the final linear layer combined with softmax activation to produce the $T \times s$ probability distribution matrix.

Embedding the discrete input sequence with T input tokens is done with a learned lookup table, which maps the discrete sequence tokens to d_{model} -dimensional real-valued vectors. To inject inductive bias about the structure of the sudoku puzzle, our model learns four separate input embeddings. The digit embedding matrix $\mathbf{W}_{E_d} \in \mathbb{R}^{s+1 \times d_{model}}$ learns to map digits 0 – s (including the special 0-digit for empty cells) in the puzzle sequence to d_{model} -dimensional real-valued vectors. The row, column and box embedding matrices $\mathbf{W}_{E_r}, \mathbf{W}_{E_c}, \mathbf{W}_{E_b} \in \mathbb{R}^{s \times d_{model}}$ learn to map the row, column and box of each token in the puzzle sequence to d_{model} -dimensional real-valued vectors. The final sudoku embedding of each sudoku puzzle $\mathbf{E} \in \mathbb{R}^{T \times d_{model}}$ is a sum of the digit, row, column and box embeddings $\mathbf{E}_{sudoku} = \mathbf{E}_d + \mathbf{E}_r + \mathbf{E}_c + \mathbf{E}_b \in \mathbb{R}^{T \times d_{model}}$.

Transformer encoder block Each of our transformer encoder blocks f_{θ_l} includes all components detailed in the original Transformer paper [9].

The first transformer block f_{θ_0} takes the sudoku embedding matrix $\mathbf{E}_{sudoku} \in \mathbb{R}^{T \times d_{model}}$ as input and produces a refined set of embeddings / hidden representations $\mathbf{H}^0 \in \mathbb{R}^{T \times d_{model}}$ through applying multi-head self-attention and a learned nonlinear transformation to the embeddings [9]. Briefly, self-attention learns to project the embeddings of the entire input puzzle sequence into three separate query (Q), key (K), and value (V) matrices $Q, K, V \in \mathbb{R}^{T \times d_k}$, each of which is obtained through three different learned linear projections $W_Q, W_K, W_V \in \mathbb{R}^{d_{model} \times d_k}$ each applied to the same $T \times d_{model}$ input puzzle embedding. Self-attention then computes the similarity scores (dot product) between the high-dimensional embeddings of every token (sudoku cell) in Q to every token in K . These similarity scores are then scaled and normalized to produce $\text{softmax}(\frac{QK^T}{\sqrt{d_k}}) \in \mathbb{R}^{T \times T}$. This $T \times T$ matrix dictates how much ‘attention’ each cell on the sudoku board should pay to all other cells on the board. Finally the output of self-attention is a attention-weighted sum of the ‘values’ V [9]:

$$\text{Self-Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V \in \mathbb{R}^{T \times d_k}$$

This mechanism is the core of why we chose to work with Transformer models for this problem. Over the course of training, the network learns such embedding and projection matrices, that the attention scores become informative for how to weigh all the clue cells (more precisely the embeddings of clue cells) to produce accurate predictions for the empty cells.

Next, I describe some implementation details. In single-headed attention, $d_k = d_{model}$, whereas in multi-headed attention (MHA) the whole self-attention described above is performed in h attention heads in parallel and thus for each attention head $d_k = d_{model}/h$. At the end of MHA, the $T \times d_k$ results are concatenated to $T \times hd_k$ and passed through a linear layer $\mathbf{W}_{out} \in \mathbb{R}^{hd_k \times d_{model}}$ that ensures that the output has the shape $T \times d_{model}$ in case d_k is chosen differently than described above.

Finally, following MHA, the embeddings of each token (sudoku cell) are enriched by being individually passed through a small two-layer multi-layer perceptron (MLP) that performs a nonlinear transformation on each d_{model} -dimensional embedding vector using the Gaussian Error Linear Unit function [10].

Our Transformer applies traditional deep learning tricks from the original paper [9], like residual connections [11] and layer normalizations [12]; as well as,

a more recent optimization by using rotary positional embeddings on the Q and K matrices [13].

Subsequent transformer blocks f_{θ_l} take the hidden representation produced by the previous transformer block $\mathbf{H}^{l-1}$, refine it further and produce the next hidden representation $\mathbf{H}^l$

$$\mathbf{H}^l = f_{\theta_l}(\mathbf{H}^{l-1})$$

Linear classification layer The stack of L transformer blocks, produces a final hidden representation $\mathbf{H}^L$. We apply a single linear layer $\mathbf{W}_{logit} \in \mathbb{R}^{d_{model} \times s}$ to each cell’s embedding in $\mathbf{H}^L$ which maps the embedding dimension to logits for the s digit classes we are trying to predict for each cell. We turn the logits into probabilities for each digit per cell $\hat{\mathbf{Y}}$ by applying a softmax over the digit class dimension.

$$\hat{\mathbf{Y}} = \text{softmax}(\mathbf{H}^L \mathbf{W}_{logit}) \in \mathbb{R}^{T \times s}$$

At test time, we obtain the predicted solution vector $\hat{\mathbf{y}}$ by choosing for each cell the digit with the highest probability: $\hat{\mathbf{y}} = \arg \max \hat{\mathbf{Y}} \in \{1, 2, \dots, s\}^{T=s^2}$.

5.3 Training strategy

Loss function We define the minimization objective $\mathcal{L}$ as a combination of a supervised classification term and a structural constraint-regularization term. The supervised loss is the standard cross entropy loss on the predicted digit distribution and the ground truth solution for each cell $\mathcal{L}_{CE}$. We introduce a custom sudoku constraint loss which penalizes violations of the row, column and box constraints $\mathcal{L}_{sudoku}$.

$$\mathcal{L}(\hat{\mathbf{Y}}, \mathbf{y}) = \mathcal{L}_{CE}(\hat{\mathbf{Y}}, \mathbf{y}) + \lambda \mathcal{L}_{sudoku}(\hat{\mathbf{Y}}) \in \mathbb{R}$$

$\mathcal{L}_{CE}$ is the main optimization objective and encourages the network to assign a high probability mass to the correct digit in each cell. We found that $\mathcal{L}_{CE}$ evaluated on the full board leads to better results than $\mathcal{L}_{CE}$ evaluated on the empty cell predictions only. $\mathcal{L}_{sudoku}$ encourages the network to produce predictions globally consistent with the row, column and box sudoku constraints (only violations of sudoku constraints from the predictions for empty cells accumulate in this term). λ is a scaling hyperparameter and controls the importance of the sudoku constraint loss, relative to the primary cross-entropy objective.

Training loop Our Transformer was implemented and trained using the PyTorch auto-differentiation deep learning library [14]. Our model accepts flexibly-sized batches of puzzles as input and is trained using minibatch stochastic gradient descent. We used the Adam optimizer [15] with decoupled weight decay [16] (`torch.optim.AdamW`) with default hyperparameters.

5.4 Results

Here we review the performance of different versions of our Transformer networks on full-board accuracy $acc_B(\hat{\mathbf{y}}, \mathbf{y})$ and per-cell accuracy $acc_C(\hat{\mathbf{y}}, \mathbf{y})$ on validation puzzles unseen by the model in training. We interpret the results relative to two base-case scenarios: 1) Random performance, 2) Memorizing the training set. In a randomly performing network, we would expect $acc_C(\hat{\mathbf{y}}, \mathbf{y}) \approx \frac{1}{9} \approx 0.11$. Because of the enormous number of possible boards, a random network should produce $acc_B(\hat{\mathbf{y}}, \mathbf{y}) \approx 0$. In case our network was overfitting and memorizing the training set, we would expect to see a large disparity between the training set and (unseen) validation set accuracies.

We do not explicitly review run-time, because that is an obvious advantage of a neural network over SAT solvers and EA’s. The number of puzzles our network can instantaneously solve in parallel (the batch size of input given to the model) is only limited by our GPU capacity ($\approx 2^{10}$ puzzles on a standard laptop).

Curriculum learning Initial training runs on full sudoku puzzles showed that model performance across many hyperparameter regimes never exceeded $acc_B = 2.5\%$ and $acc_C = 55.5\%$ (not shown). This was likely because expecting the model to learn the full game of sudoku from puzzles with most of the cells being empty was too difficult. We attempted to fix this by training on toy problems, where we randomly revealed different parts of each puzzle solution in each epoch, to control how many cells (“empties”) the network had to predict at each stage. Our idea was to start with predicting a few empties and increase the difficulty over training epochs until the model learned to solve full puzzles.

Figure 4 shows a subset of training runs with different model parameters and curriculum schedules. It is beyond the scope of this report to provide a detailed analysis of the effects of individual hyperparameters (at least 15 highly influential degrees of freedom) on the performance of the model over all the (50+) experimental runs. Instead, we discuss general insights that emerged across hyperparameter regimes. Models were trained on 3% of the sudoku-extreme training dataset (103K puzzles of all difficulties), and tested on 10K unseen test puzzles. Most runs explored the (30-35-40-45-50-full) curriculum schedule, where the number indicates the number of empties at each curriculum stage. We explored co-varying the batch size, learning rate, number of epochs and sudoku loss weight λ with the curriculum schedule.

First, model performance generalized near perfectly from training (Appendix A.1) to unseen data (Figure

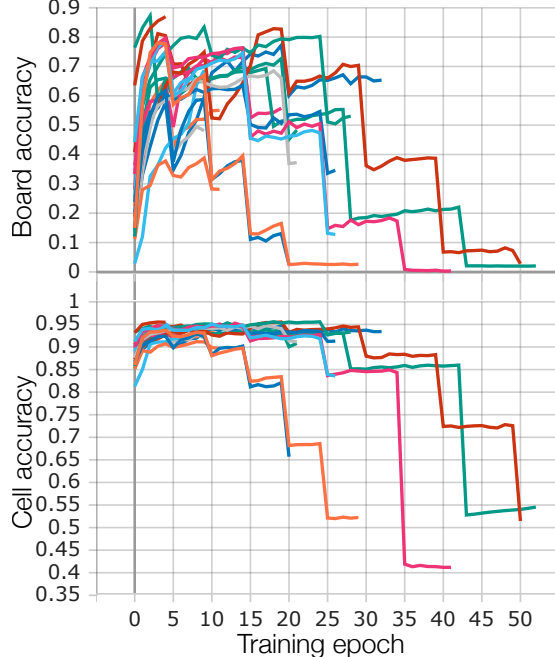


Figure 4: Board and cell accuracy of curriculum-trained models on unseen validation puzzles. Lines show separate runs with different hyperparameters.

4), indicating real learning of the rules and solution strategies of sudoku, instead of memorizing the training set. Since each retrieval of each training sample contained a different random subset of empties, this forced the model to capture general patterns in the puzzles instead of memorizing the training set.

Secondly, we were pleased to find that most hyperparameter configurations achieved $acc_B > 70\%$ on the 30 and 35 empties within a few epochs (although acc_B plummeted in later epochs at 40+ empties curricula). Importantly, acc_C remained around 90% for many models, even as acc_B started to fall, and remained far above chance level in later epochs when acc_B plummeted. This suggests that for a long time, the model was predicting most empties correctly and the rare mistakes it made were an insufficient drive to perfectly learn the rules.

Finally, despite our best effort and extensive testing of hyperparameter settings, curriculum learning did not help in transitioning to sudokus with fewer clues (Figure 4, notice abrupt drops in accuracy at each curriculum change). Regardless of the number of training epochs spent on the “harder curricula” (40+ empties), the models were unable to recover from the second drop in performance when transitioning from 35 to 40 empties (the first drop in performance at transition from 30 to 35 empties could be absorbed). Later transitions to emptier and emptier boards con-

sistently brought performance to unrecoverably low levels ($acc_B < 10\%$ for 50 empties and $acc_B \approx 2\%$ for full puzzles, same as without curriculum training).

Recurrent Transformers After struggling with curriculum learning, we discovered a paper that had successfully trained Recurrent Transformers on sudoku and other CSPs[17]. The main difference from our feed-forward approach of stacking transformer blocks after each other, is that the recurrent approach repeatedly feeds the output of a single transformer block back to itself as input a fixed number of times.

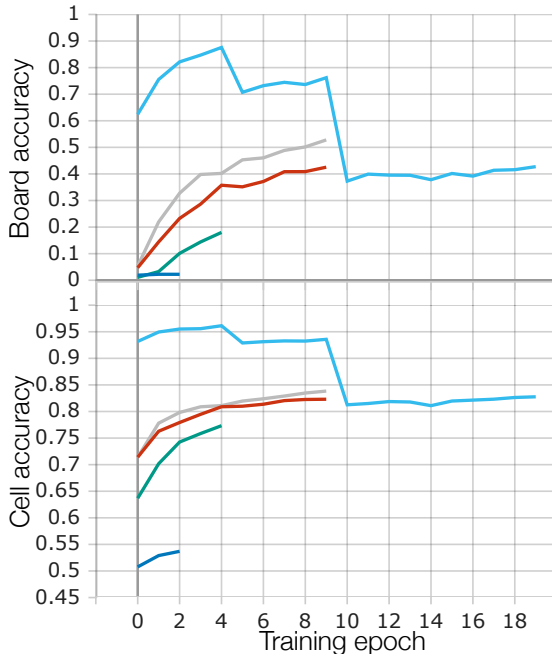


Figure 5: Board and cell accuracy of Recurrent Transformers on unseen validation puzzles with 50 empties. Lines show runs with different recursion depths.

With each feedback step, the model refines its previous hidden representations using the same set of model parameters. This mimics the reasoning approach, which requires iterative refinement of internal representations to arrive at the solution. Because the implementation was trivial (feed $\mathbf{H}^L$ back to f_{θ_0} and the chain of transformer blocks r times, before proceeding to the linear classification layer), we explored this modification of our model.

Figure 5 shows results of training the same Transformer model (learning rate = $5e-4$, batch size = 64, $\lambda = 1e-3$, $d_{model} = 150$, $h = 3$, $L = 5$) with different number of recurrent feedback cycles r on 50-empties sudoku problems (31 clues). Comparing the red ($r = 3$, 50-only) and light-blue ($r = 3$, 40-45-50 curriculum) lines shows that curriculum learning

also does not work for recurrent transformers, as the 50-only model with the same recursion depth reaches the same performance as the curriculum model that first learns to solve easier problems with high accuracy. Nevertheless, board accuracies achieved by the $r = 3$ recurrent curriculum model at 40 and 45 empties (90% and 75%) are far above the best accuracies observed without recurrence, where the models collapsed at 40+ empties. Similarly, at 50 empties the best model (gray, $r = 4$) achieved $acc_B \approx 55\%$ and $acc_C \approx 83\%$ in only 10 epochs of training, significantly surpassing the sub-10% acc_B of non-recurrent models. Finally, we found that increasing recurrence past $r = 5$ (dark-blue), does not improve performance on 50 empties relative to non-recurrent models, whereas increasing recursion depth from 2 (green) to 3 (red) to 4 (gray) leads to steady improvements.

6 Discussion

Here we investigated three different approaches to solving the logical puzzle of sudoku. We find that while SAT solvers are guaranteed to eventually find the sudoku solution, the act of encoding the problem into propositional variables and navigating the resulting search space can scale poorly to larger and more highly constrained puzzles. Nevertheless, modern SAT solving techniques such as CDCL substantially mitigate this issue and enable fast reasoning even on large and highly constrained problems. Furthermore, the propositional encoding is not only a weakness, but also a strength of SAT solvers, since it enables these solvers to be applied to any general CSP, regardless of the problem domain.

Evolutionary algorithms (EAs) approach the problem from the opposite direction: instead of reasoning symbolically, they explore the space of complete candidate boards using stochastic operators and conflict-based fitness functions. In theory, this allows them to exploit structural regularities in the solution space, and EAs have been shown to achieve high accuracy on classical sudoku. In our experiments, however, the EA did *not* outperform SAT in runtime, contrary to our initial hypothesis. While our EA solved most classical puzzles, it remained slower and less reliable than CDCL, and its performance collapsed in the more constrained non-consecutive sudoku setting.

While our Transformer neural network did inevitably outperform the SAT and EA algorithms on runtime (not shown), due to its highly optimized parallel nature and being able to infer solutions to large batches of puzzles at once, this property would only be useful when paired with high accuracy. Our SudokuTransformer severely struggled to learn to solve

full sudoku puzzles, which can have as few as 17 clues, thereby failing massively in comparison with SAT and EAs on the full problems. However, training the model on toy puzzles with more clues (generated by randomly revealing some of the empty cells) provided convincing evidence that the neural network was able to genuinely learn the rules and solution strategies of sudoku purely through exposure to a large number of puzzles. The performance achieved on the toy training examples generalized exceptionally well to previously unseen toy examples, providing further evidence of the model capturing nontrivial sudoku rules in its parameters. Our curriculum training approach to gradually scale up the toy problem difficulty by slowly revealing fewer and fewer clues did not lead to better accuracy than simply training on the more difficult problems for the same number of epochs. This may suggest that the curriculum stages we used were too coarse (steps of 5 less clues per curriculum), and changed the data distribution too drastically for the network to recover from those shifts. It is possible that using more sophisticated curriculum scheduling with gradual blending of the difficulties into one another would yield better results. Furthermore, a few short but revealing runs suggested that using a larger dataset led to better performance, across toy problem difficulties. Because of compute and time constraints, we could not use more than 100K training samples for extended runs, but we suspect that curriculum training could be significantly improved by scaling the dataset size with the increasing problem difficulty. One reason might be, that to learn the more advanced solution strategies required in problems with fewer clues, the network needs to see many more examples. Finally, by far the largest boost to the transformer performance came from introducing recurrent re-processing of the model’s own output, both during training and inference. Our simple implementation of a Recurrent Transformer was still insufficient for full sudoku puzzles, but it pushed the performance categorically beyond what was possible with a purely feed-forward Transformer. We consider this compelling evidence that a more refined implementation of recurrence (for instance [17]), applied to a larger dataset could lead to a full-puzzle-capable models. Conceptually, the addition of recurrence allowed the model to perform a sort of ‘search’ on each puzzle by refining its own internal representation of each puzzle multiple times. While this considerably slowed down performance (by a factor of the recursion depth), the model could still accept arbitrarily large batch sizes and process them much quicker than a SAT solver. However, with enough recursive calls, the performance would eventually slow down to, and

potentially below, SAT level, simply because of the sequential nature of recurrent feedback. This sequential “search” performance bottleneck is ultimately the same limitation that SAT solvers and EAs run into, although Transformers and neural networks in general are well-equipped to provide highly informative priors for such search. Indeed using the output of (Recurrent) Transformer networks with highly-optimized SAT solvers could be a fruitful pursuit not just for Sudoku, but for CSPs in general.

Our results highlight a clear distinction in how symbolic solvers, evolutionary methods and deep learning models handle constraint propagation and the dense interdependence of sudoku constraints. Among the SAT solvers, CDCL’s clause learning, non-chronological backtracking and restart strategies enable rapid convergence even on tightly constrained variants, substantially outperforming the more traditional DPLL. On the other hand, the evolutionary algorithm occasionally stagnates in regions of the search space that are locally consistent but globally invalid. The Transformer model showed promising behavior on sudoku toy-problems in our experiments, highlighting the ability of neural networks to learn logical rules and simple solution strategies. Future research should explore how a fully-optimized version, trained on larger datasets and equipped with deeper recurrence, would compare. For now, symbolic reasoning remains the most reliable route for solving tightly constrained sudoku, though the landscape may shift as neuro-symbolic and learned inference methods continue to mature.

References

- [1] Inês Lynce and Joël Ouaknine. Sudoku as a sat problem. *International Conference on AI and Mathematics*, 2006.
- [2] Tjark Weber. A sat-based sudoku solver? Technical report, Technische Universität München, 2005.
- [3] Zachary Kincaid. Sat handbook - cdcl. <https://www.cs.princeton.edu/~zkincaid/courses/fall18/readings/SATHandbook-CDCL.pdf>, 2018.
- [4] Xiu Qin Deng and Yong Da Li. A novel hybrid genetic algorithm for solving sudoku puzzles. *Optimization Letters*, 7:241–257, 2011.
- [5] P. R. Amil and T. Mantere. Solving sudokus by evolutionary algorithms with pre-processing. In *Recent Advances in Soft Computing: Proceedings of MENDEL 2017*, volume 837, pages 3–15. Springer, 2019.
- [6] Yuji Sato, Naohiro Hasegawa, and Mikiko Sato. Acceleration of genetic algorithms for sudoku solution on many-core processors. In *Human-Competitive Machine Intelligence*, pages 243–258. Springer, 2015.
- [7] Liam Davis and Tairan Ji. Evaluating sat and smt solvers on large-scale sudoku puzzles. *arXiv preprint arXiv:2501.08569*, 2025.
- [8] Kurt Hornik, Maxwell Stinchcombe, and Halbert White. Multilayer feedforward networks are universal approximators. *Neural networks*, 2(5):359–366, 1989.
- [9] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- [10] D Hendrycks. Gaussian error linear units (gelus). *arXiv preprint arXiv:1606.08415*, 2016.
- [11] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.
- [12] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.
- [13] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.
- [14] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems*, 32, 2019.
- [15] Diederik P Kingma. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- [16] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*, 2017.
- [17] Zhun Yang, Adam Ishay, and Joohyung Lee. Learning to solve constraint satisfaction problems with recurrent transformer. *arXiv preprint arXiv:2307.04895*, 2023.

A Appendix

A.1 Curriculum Transformer training results

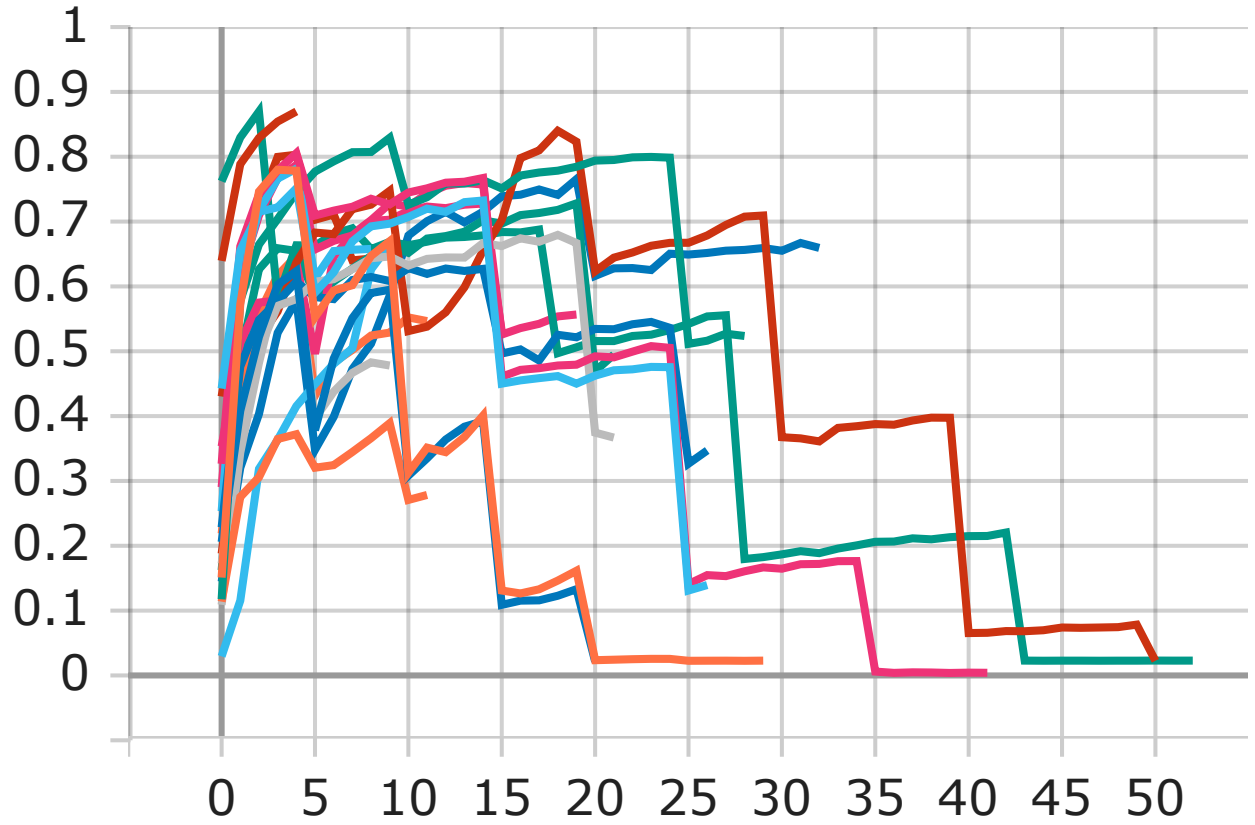


Figure 6: Board accuracy of curriculum-trained models on training puzzles. X-axis shows training epochs, y-axis shows board accuracy. Lines show separate runs with different hyperparameters.

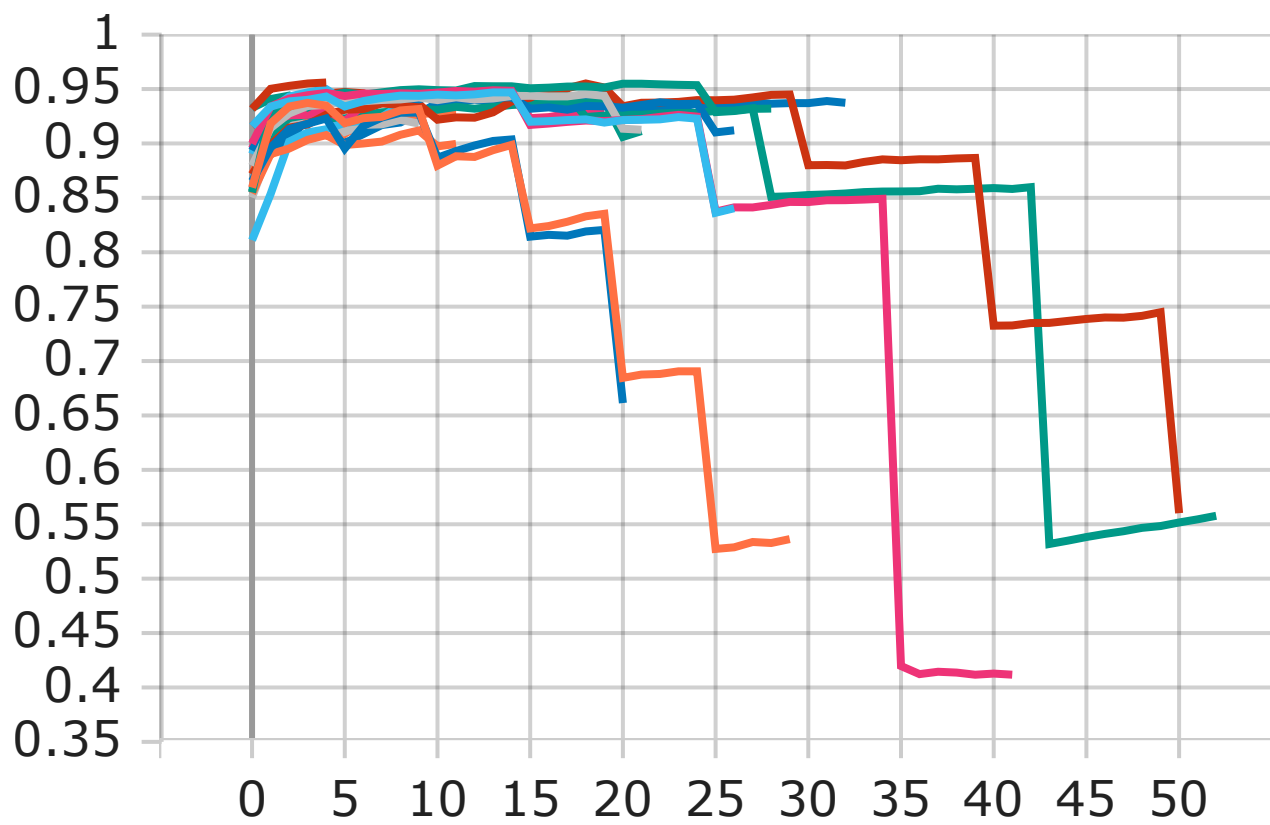


Figure 7: Cell accuracy of curriculum-trained models on training puzzles. X-axis shows training epochs, y-axis shows cell accuracy. Lines show separate runs with different hyperparameters.